

```
#include <bits/stdc++.h>
#include <unistd.h>
#include <sys/mman.h>
#include <signal.h>
using namespace std;

// ----- Guard Page + 재귀 시뮬레이션 -----

// 페이지 크기
size_t PAGE_SIZE;

// 2페이지 블록: 첫 페이지 일반, 두 번째 페이지 Guard Page
char *simulated_stack = nullptr;

// SIGSEGV 핸들러
void handle_sigsegv(int sig)
{
    cerr << "[Guard Page] Access detected! Signal: " << sig << "\n";
    exit(1); // 안전하게 종료
}

// Guard Page 초기화
void init_guard_page()
{
    PAGE_SIZE = sysconf(_SC_PAGESIZE);

    // aligned malloc으로 페이지 경계 맞춤
    simulated_stack = (char *)aligned_alloc(PAGE_SIZE, PAGE_SIZE * 2);
    if (!simulated_stack)
    {
        cerr << "Failed to allocate simulated stack\n";
        exit(1);
    }

    // 두 번째 페이지를 Guard Page로 설정
    void *guard_addr = simulated_stack + PAGE_SIZE;
    if (mprotect(guard_addr, PAGE_SIZE, PROT_NONE) != 0)
    {
        perror("mprotect failed");
        exit(1);
    }

    cout << "Guard Page initialized at simulated stack end\n";
}

// 재귀 테스트
void recursion_sim(long long depth = 0)
{
    size_t offset = depth * 16; // 한 단계마다 16바이트 사용
    char *ptr = simulated_stack + offset;
    ptr[0] = 42; // Guard Page 접근 시 SIGSEGV 발생
}
```

```
    if (depth % 100 == 0)
        cout << "Recursion depth: " << depth << "\n";

    recursion_sim(depth + 1);
}

int main()
{
    signal(SIGSEGV, handle_sigsegv); // SIGSEGV 핸들러 등록

    init_guard_page();

    cout << "Starting recursion simulation...\n"
         << flush;
    recursion_sim(); // 재귀 시작

    return 0;
}
```